Recurrent Neural Networks Part 2

Agenda

Recap RNNs,

LSTMs and GRUs

Generative RNNs

Recap: Word Embedding

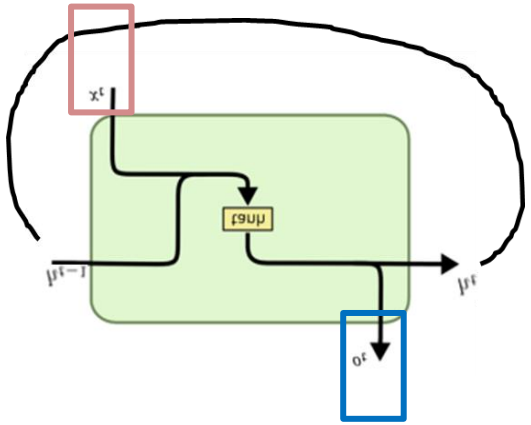
Goal:

- Allow for processing text into **numerical values**
- Capture **meaningful representations** of words
- Simplify **downstream tasks**
  - sentiment analysis (last time),
  - text summarizations,
  - generating text (today)

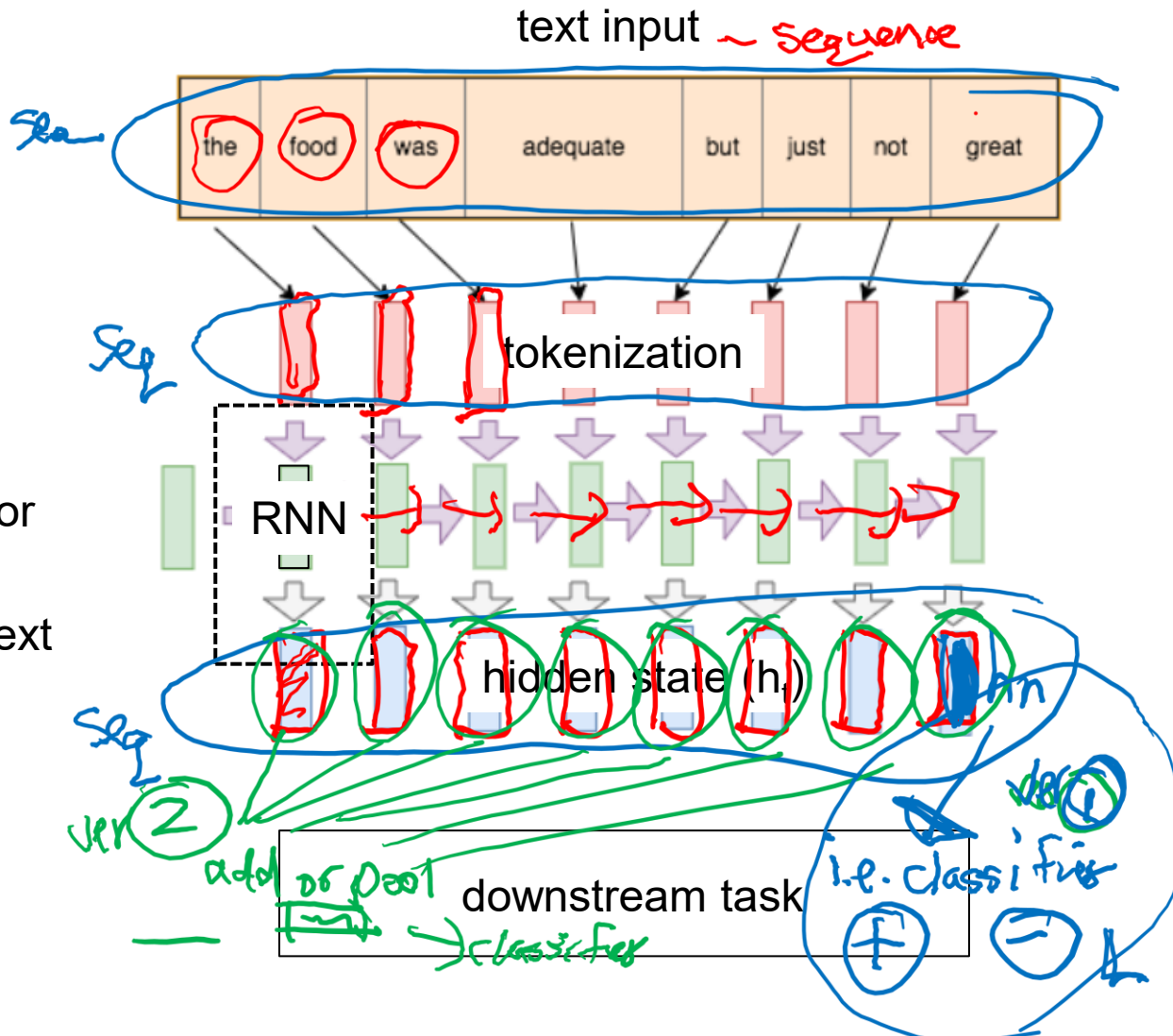
Limitation of GloVe and Word2Vec

- word-level embedding, fixed meaning independent of context, **require sentence-level embedding (or greater)**

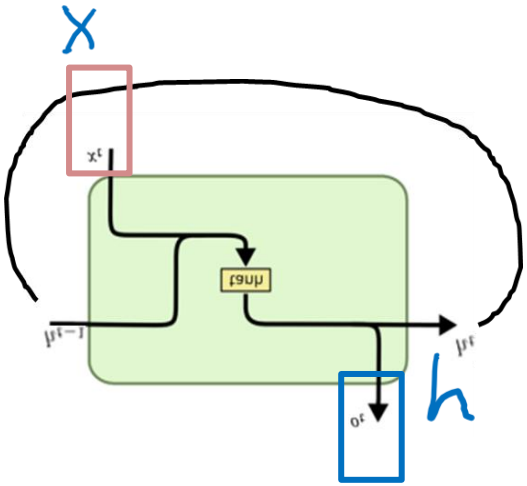
Recap: RNN Model



- Reuse the RNN module for entire length sequence
- Keeping track of the context in the hidden state (*h*)



Recap: RNN Model



- Reuse the RNN module for entire length sequence
- Keeping track of the context in the hidden state (*h*)

$$h_t = \tanh(W_{hh} h_{t-1} + W_{xh} x_t)$$

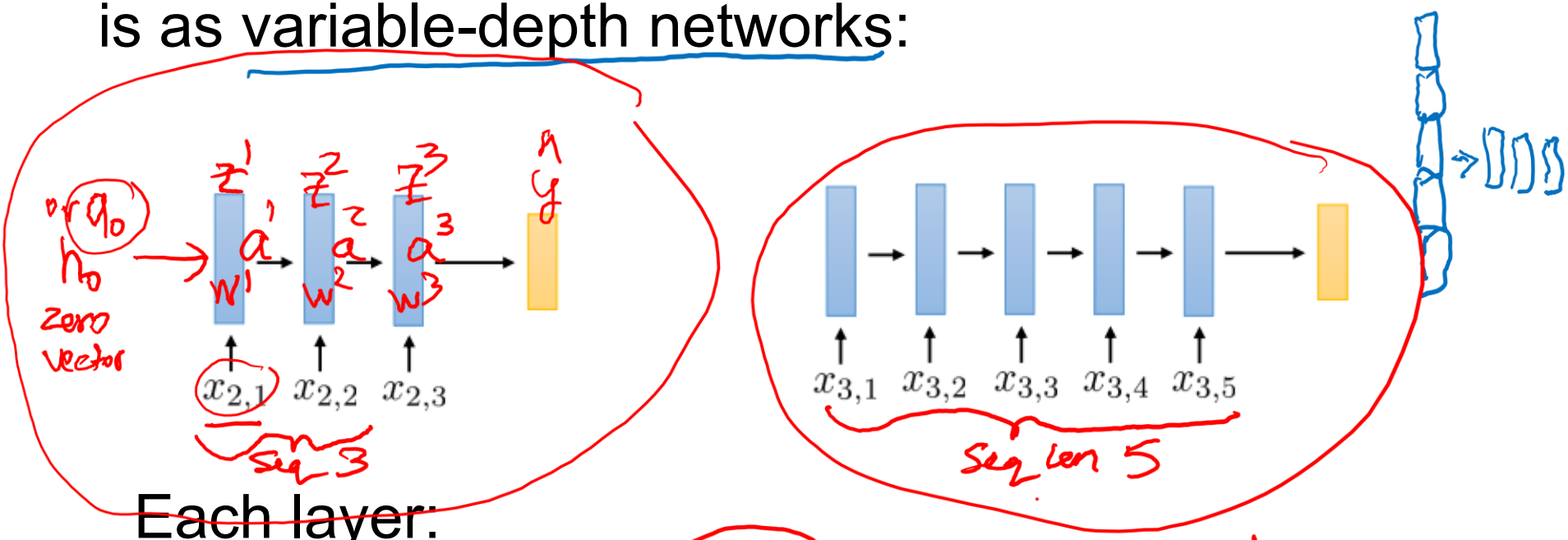
$$= \tanh\left((W_{hh} \quad W_{hx}) \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix} \right)$$

$$= \tanh\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix} \right)$$

Concat

RNNs – Another way to think about it

Another way to think of recurrent neural networks is as variable-depth networks:



Each layer:

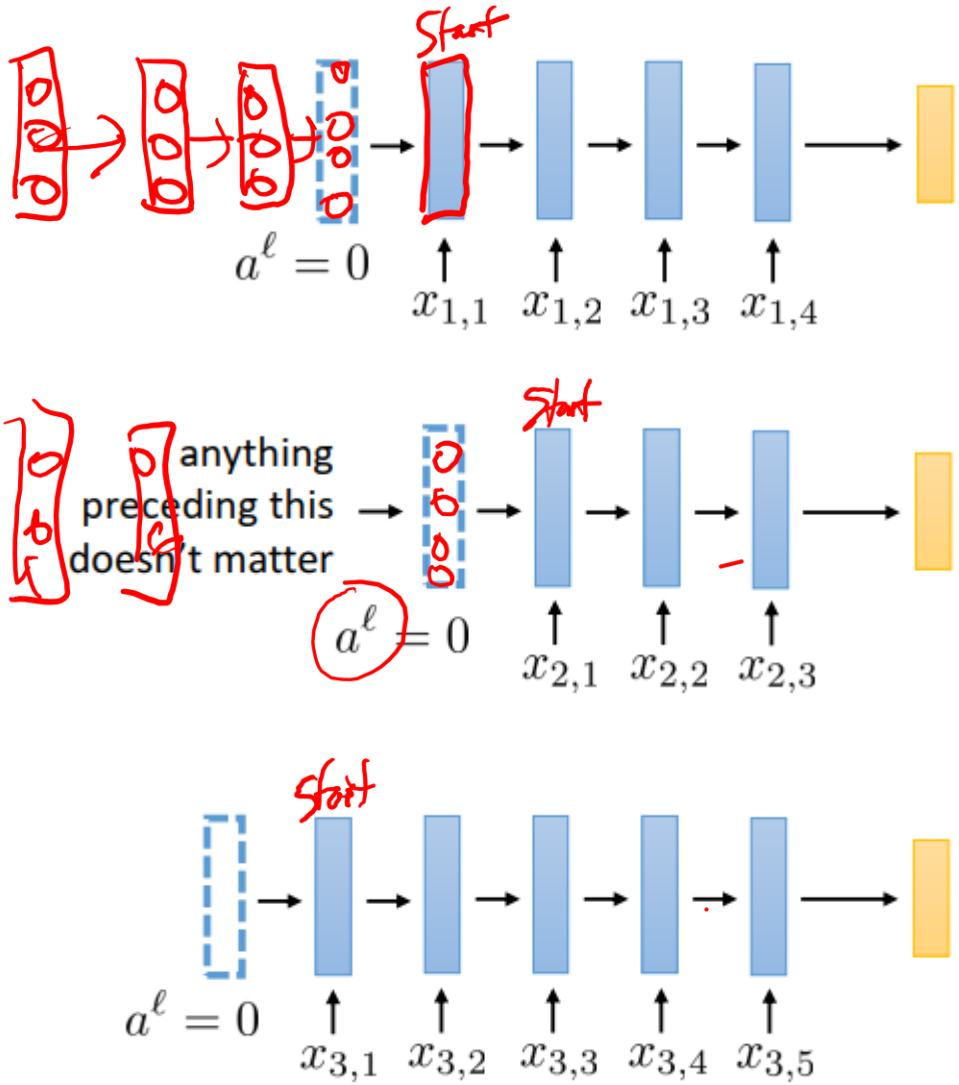
layer

$$\bar{a}^{\ell-1} = \begin{bmatrix} a^{\ell-1} \\ x_{i,t} \end{bmatrix} \qquad z^{\ell} = W^{\ell}\bar{a}^{\ell-1} + b^{\ell} \qquad a^{\ell} = \sigma(z^{\ell})$$

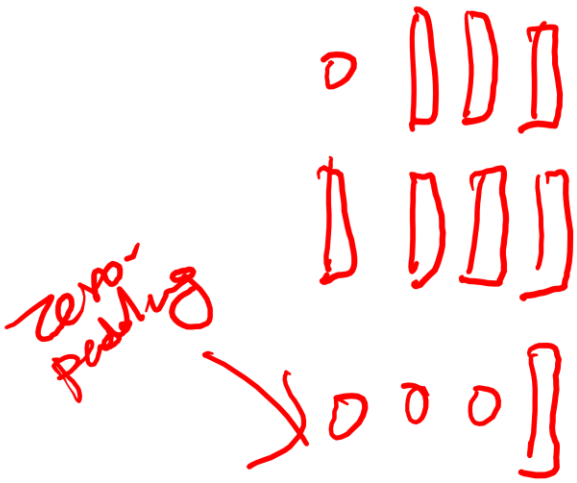
activation

Source: Sergey Levine

RNNs – Another way to think about it

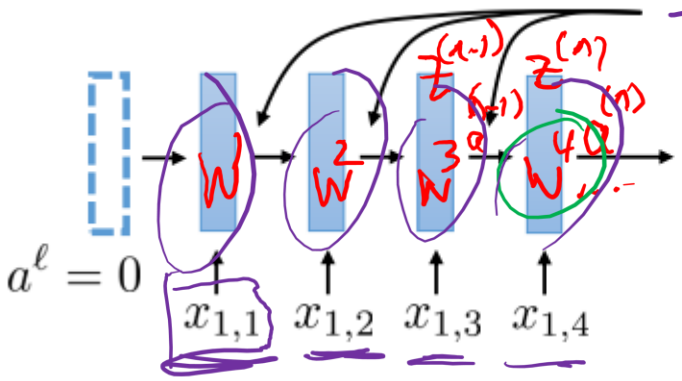


Q: How to handle variable length inputs?



Source: Sergey Levine

RNNs – Another way to think about it



W, b are same, i.e., shared at each layer

$$W = W^{(1)} = W^{(2)} = W^{(3)} \cdots$$

$$W^{\ell_i} = W^{\ell_j} \text{ for all } i, j$$

$$b^{\ell_i} = b^{\ell_j} \text{ for all } i, j$$

$$\bar{a}^{\ell-1} = \begin{bmatrix} a^{\ell-1} \\ x_{i,t} \end{bmatrix}$$

$$z^\ell = W^\ell \bar{a}^{\ell-1} + b^\ell$$

$$a^\ell = \sigma(z^\ell)$$

$$\frac{da^\ell}{dz^\ell} = ?$$

$$\frac{dz^\ell}{da^{\ell-1}} = W^\ell$$

$$\frac{dz^\ell}{dW^\ell} = \begin{bmatrix} a^{\ell-1} \\ x_{i,\ell} \end{bmatrix}$$

Q: How do we learn the weights?

$$\frac{dL}{dW^{(1)}} = \frac{dL}{d\hat{y}} \cdot \frac{d\hat{y}}{dz^{(n)}} \cdot \frac{dz^{(n)}}{da^{(n-1)}} \cdots \frac{da^{(2)}}{dz^{(2)}} \cdot \frac{dz^{(1)}}{dW^{(1)}}$$

seq length

If weights all same

$$\frac{dL}{dW} = \sum_t \frac{dL}{dW^{(t)}}$$

Source: Sergey Levine

Continue at 2:10pm

LSTM

Recap

Last Time

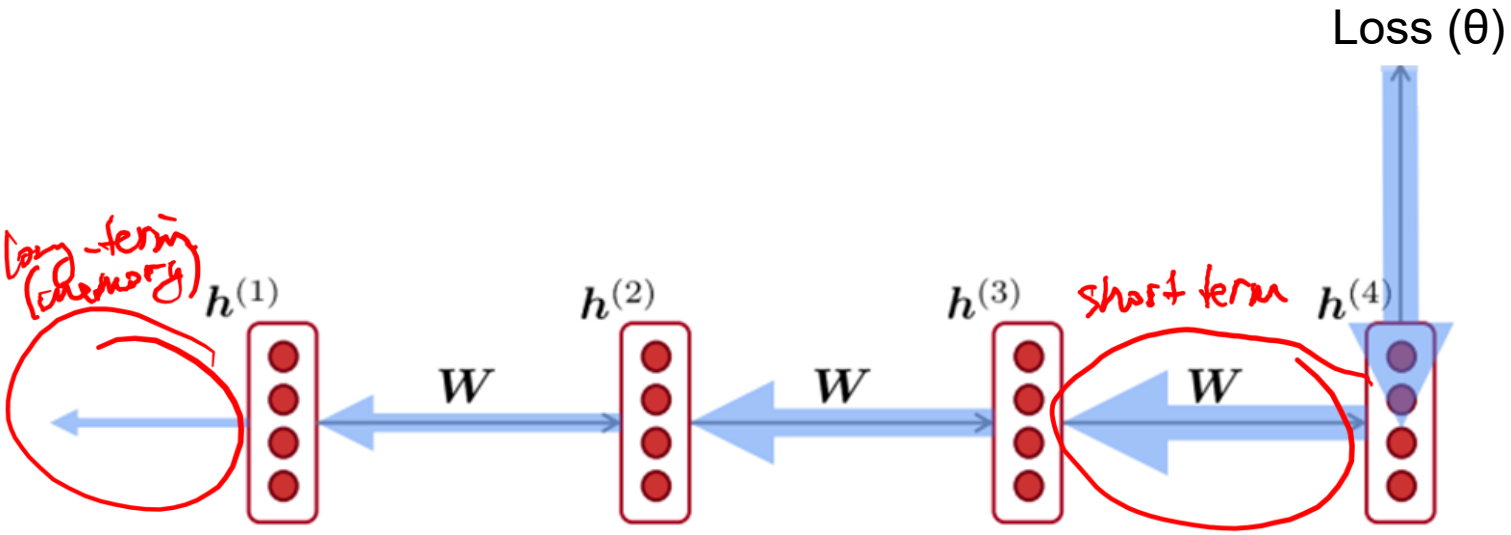
- Introduced Recurrent Neural Networks for working with sequential inputs

Today

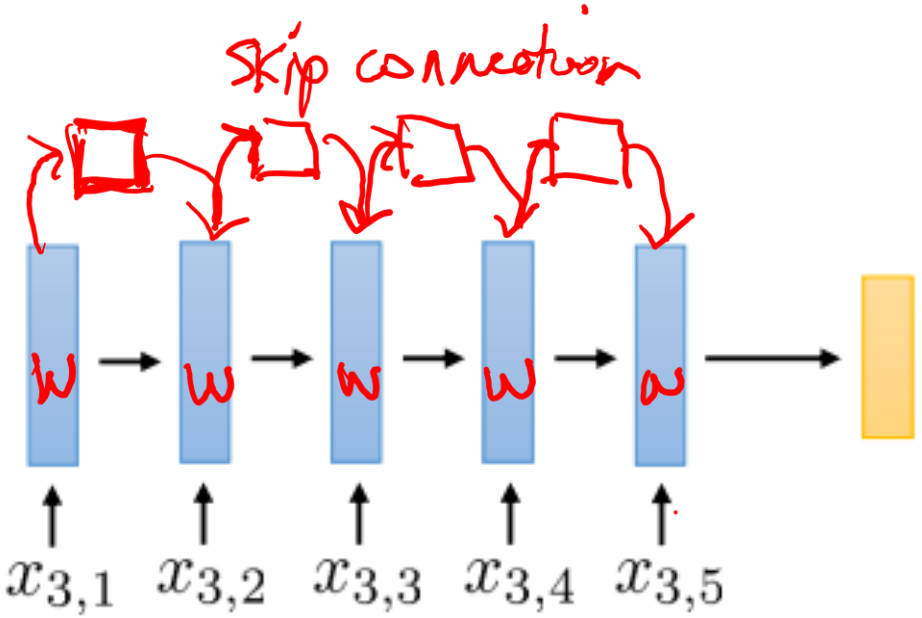
- Making gradients manageable
- … and working with sequential outputs

Vanishing/Exploding Gradients

- Recurrent Neural Networks were historically hard to train
- Good for capturing recent information, but difficulties with long-term dependencies
  - Can you see why this is the case?



Vanishing/Exploding Gradients



Q: How do we address this limitation of Recurrent Neural Networks?

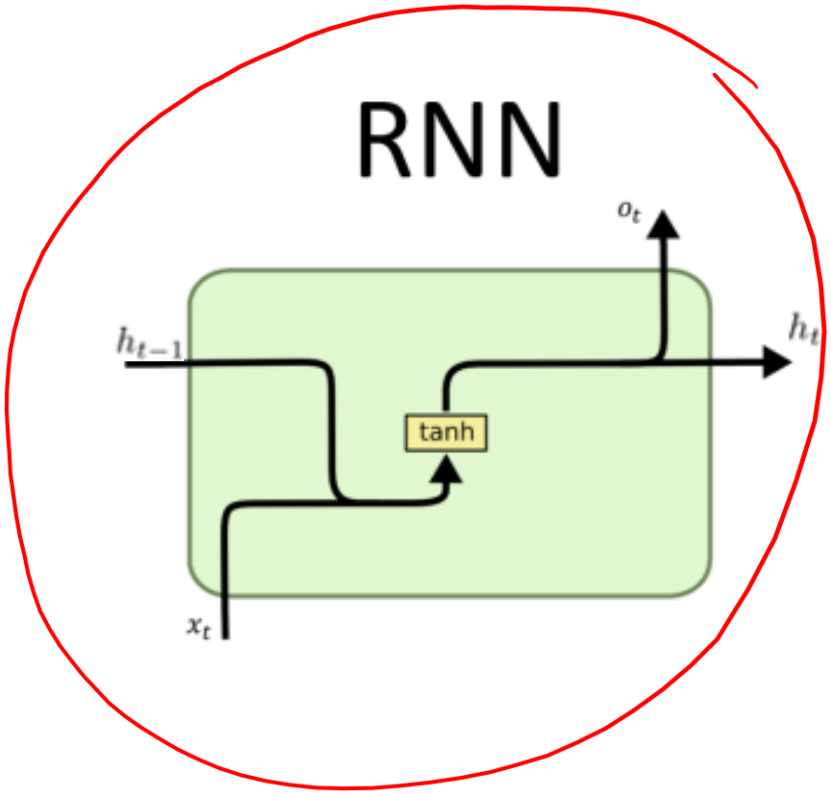
Learning Long-Term Dependencies

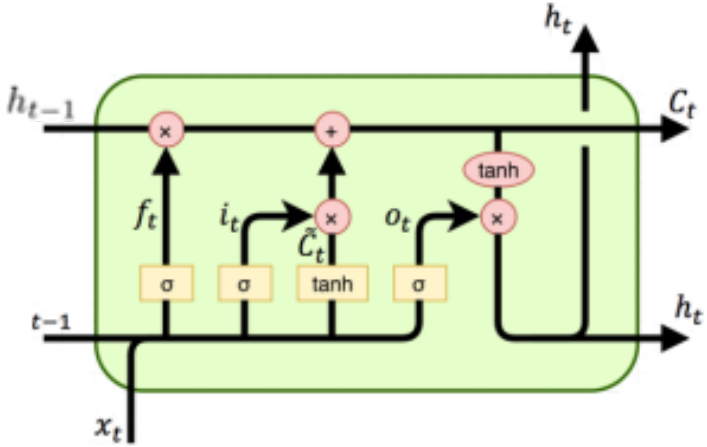
Solution

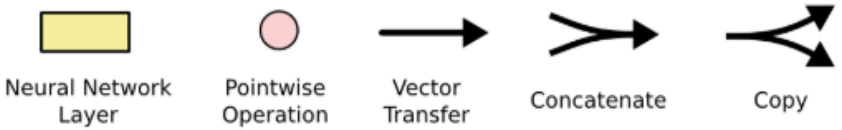
- **Long Short-Term Memory (LSTM)**
  - take advantage of an extra hidden state referred to as the **cell state**
  - gates to manage long-term memory and regulate **vanishing gradients**
  - gradient clipping resolves **exploding gradients**
  - adds more complexity

Terminology Note: *LSTMs fall under the general RNN term, and we refer to original (simple) RNNs as "vanilla RNN"*

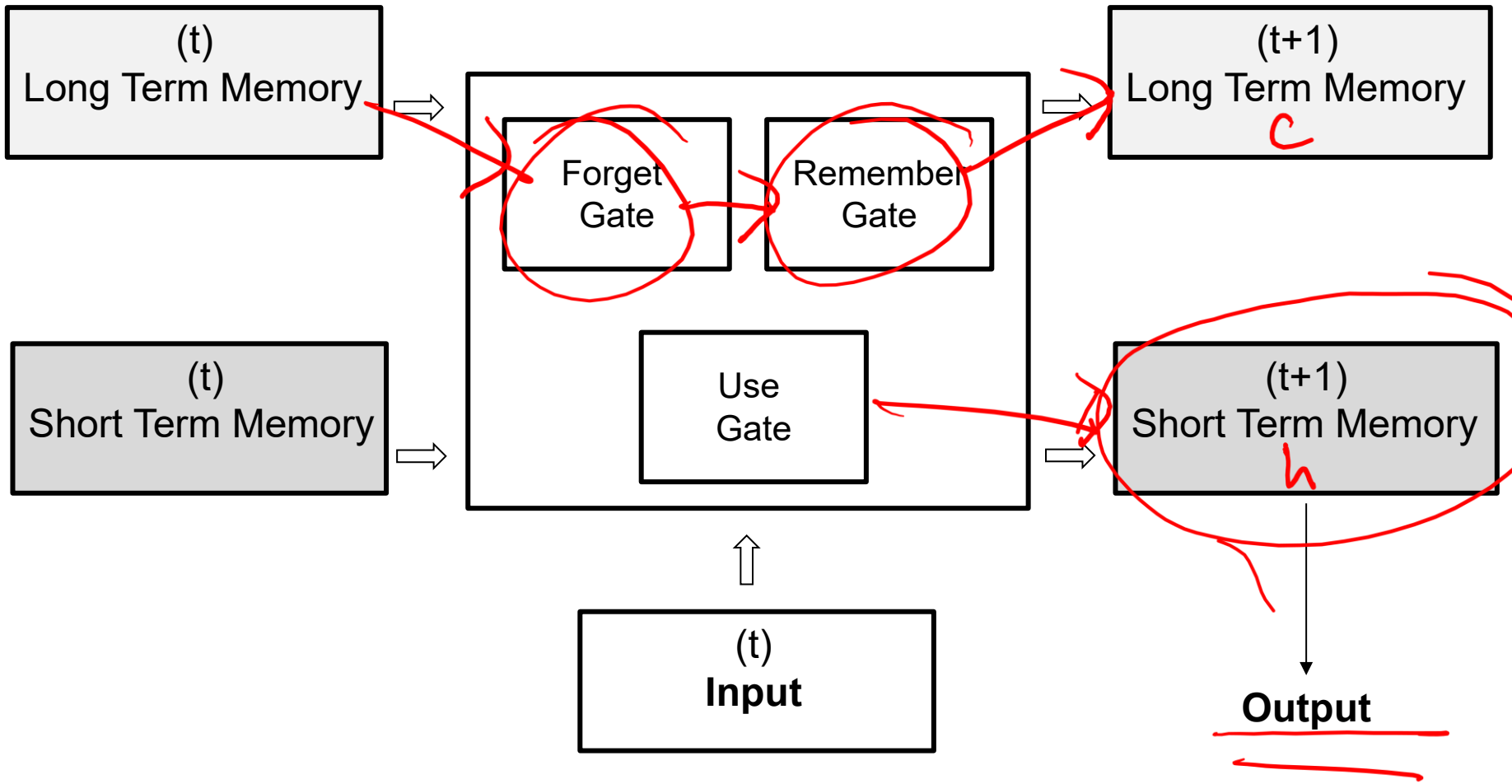
RNN vs LSTM


RNN
o_t
h_t
h_{t-1}
tanh
x_t

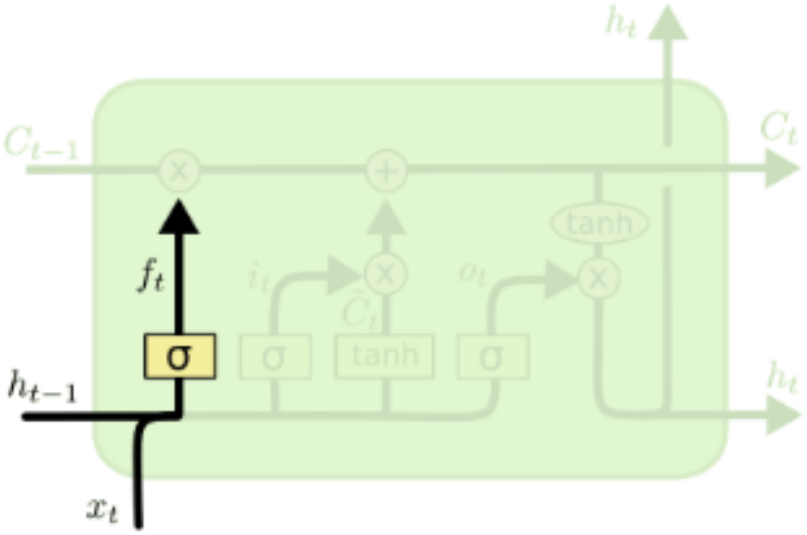

LSTM
h_t
h_{t-1}
C_t
tanh
f_t
i_t
o_t
$\tilde{C}_t$
σ
σ
tanh
σ
$t-1$
h_t
x_t


Neural Network Layer
Pointwise Operation
Vector Transfer
Concatenate
Copy

Conceptual: Long Short-Term Memory



LSTM: Forget Gate



f_t

mask → 1 0 1 1 1 0 0 1 0

cell memory ? ? ? ? ? ? ? ?

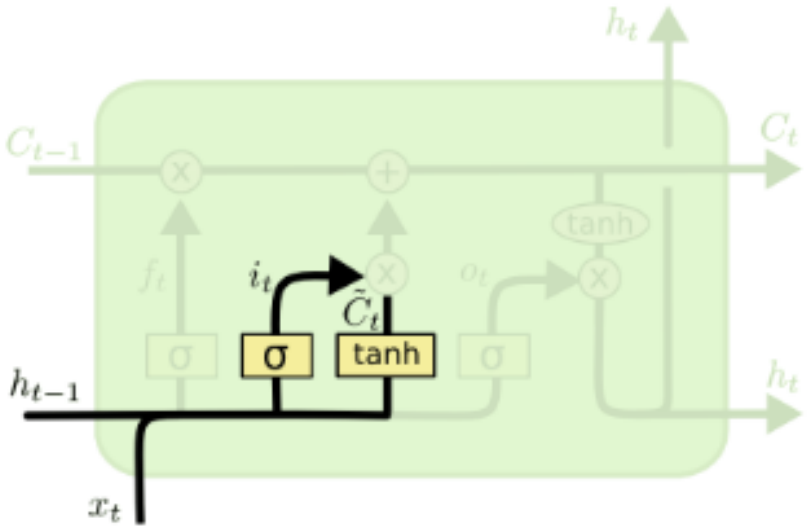
$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] + b_f\right)$$

inputs

sigmoid

[0, 1]

LSTM: Remember Gate



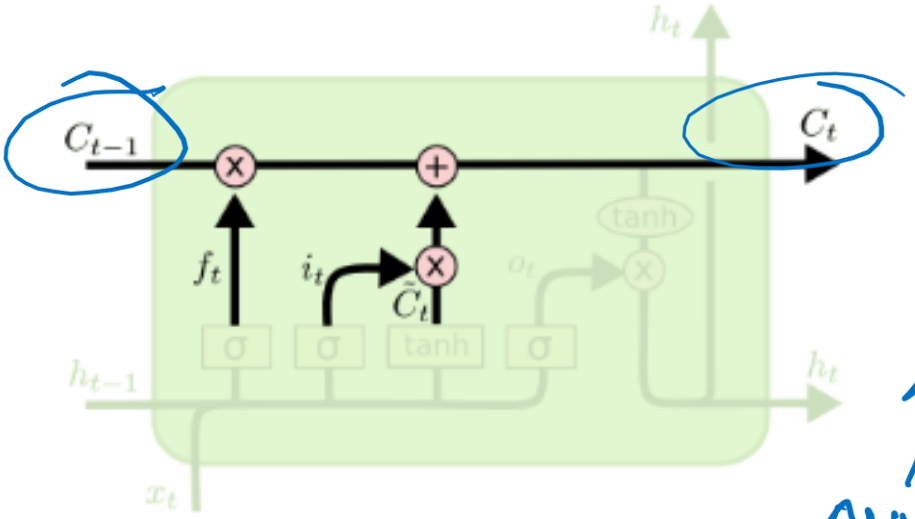
i_t remember Mask

$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] + b_i\right)$$

$$\tilde{C}_t = \tanh\left(W_C \cdot [h_{t-1}, x_t] + b_C\right)$$

New information

LSTM: Remember Gate Con't



Update of the cell state

forget mask

remember mask

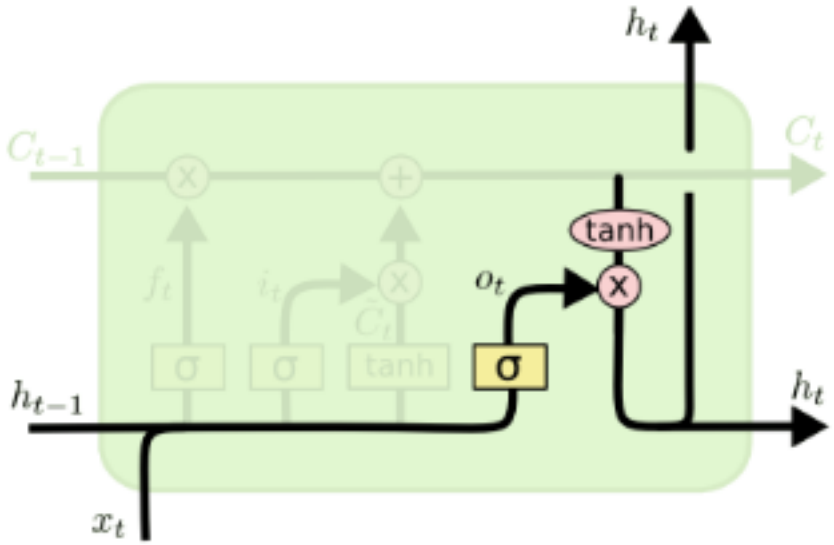
$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

past info

new info

new memory (long-term memory)

LSTM: Use Gate



$$o_t = \sigma\left(W_o\left[h_{t-1}, x_t\right] + b_o\right)$$

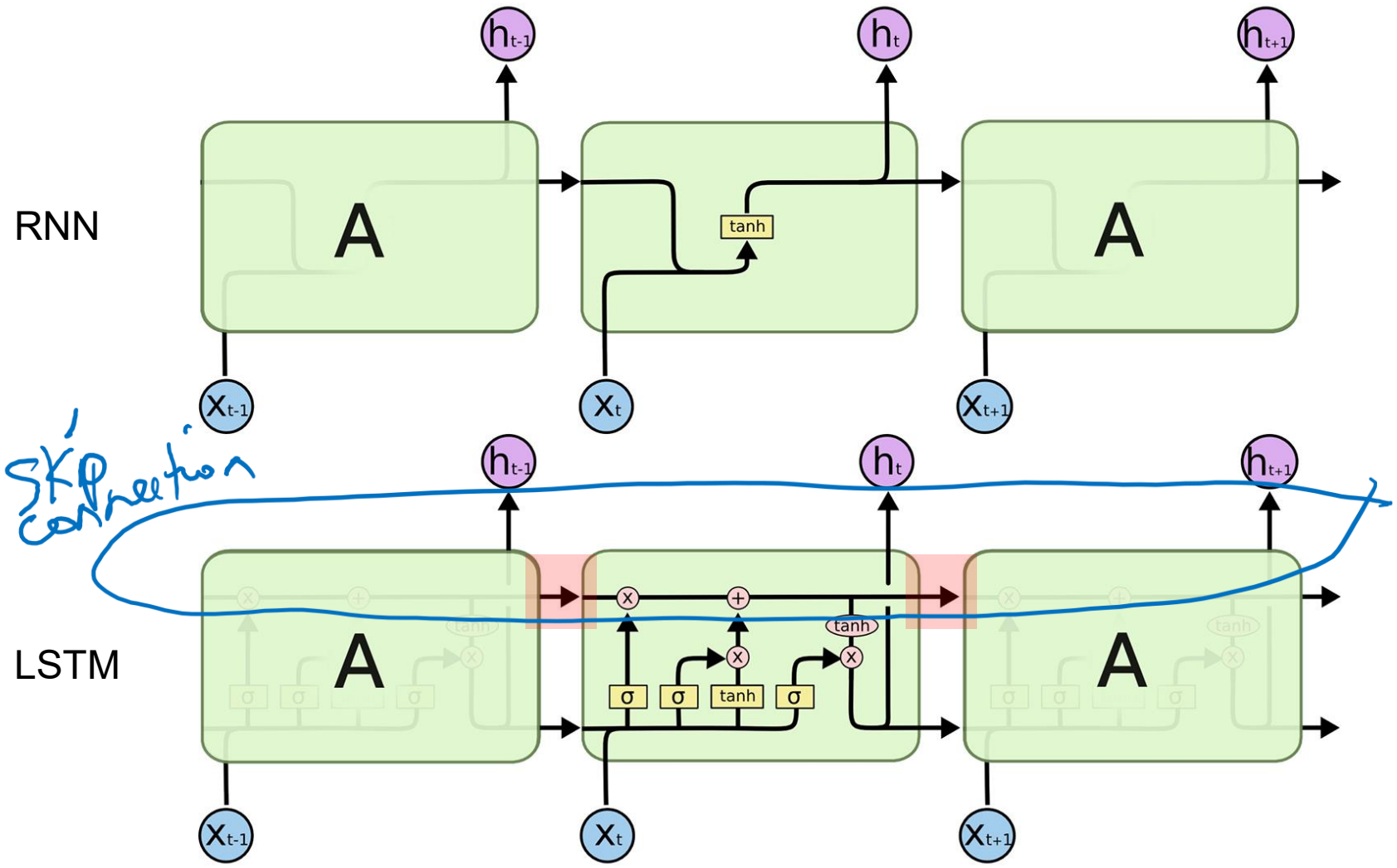
$$h_t = o_t * \tanh(C_t)$$

use mask

selecting

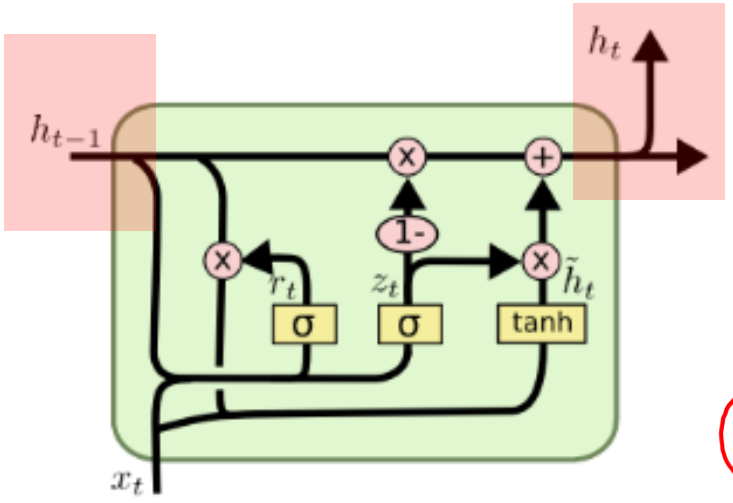
long-term memory

RNN vs LSTM


RNN
A
tanh
A
A
h_{t-1}
h_t
h_{t+1}
x_{t-1}
x_t
x_{t+1}
Skip connection
LSTM
A
σ
σ
tanh
σ
tanh
A
A
h_{t-1}
h_t
h_{t+1}
x_{t-1}
x_t
x_{t+1}

Gated Recurrent Unit

- GRU is a variant on LSTM
- Introduced in 2014, it combines gates and merges cell state and hidden state
- It is much simpler to implement and faster than standard LSTM



$$z_t = \sigma\left(W_z \cdot [h_{t-1}, x_t]\right)$$

$$r_t = \sigma\left(W_r \cdot [h_{t-1}, x_t]\right)$$

$$\tilde{h}_t = \tanh\left(W \cdot [r_t * h_{t-1}, x_t]\right)$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

PyTorch Implementation: RNN

```
class TweetRNN(nn.Module):
    def __init__(self, input_size, hidden_size, num_classes):
        super(TweetRNN, self).__init__()
        self.emb = nn.Embedding.from_pretrained(glove.vectors)
        self.hidden_size = hidden_size
        self.rnn = nn.RNN(input_size, hidden_size, batch_first=True)
        self.fc = nn.Linear(hidden_size, num_classes)

    def forward(self, x):
        # Look up the embedding
        x = self.emb(x)
        # Set an initial hidden state
        h0 = torch.zeros(1, x.size(0), self.hidden_size)
        # Forward propagate the RNN
        out, _ = self.rnn(x, h0)
        # Pass the output of the last time step to the classifier
        out = self.fc(out[:, -1, :])
        return out
```

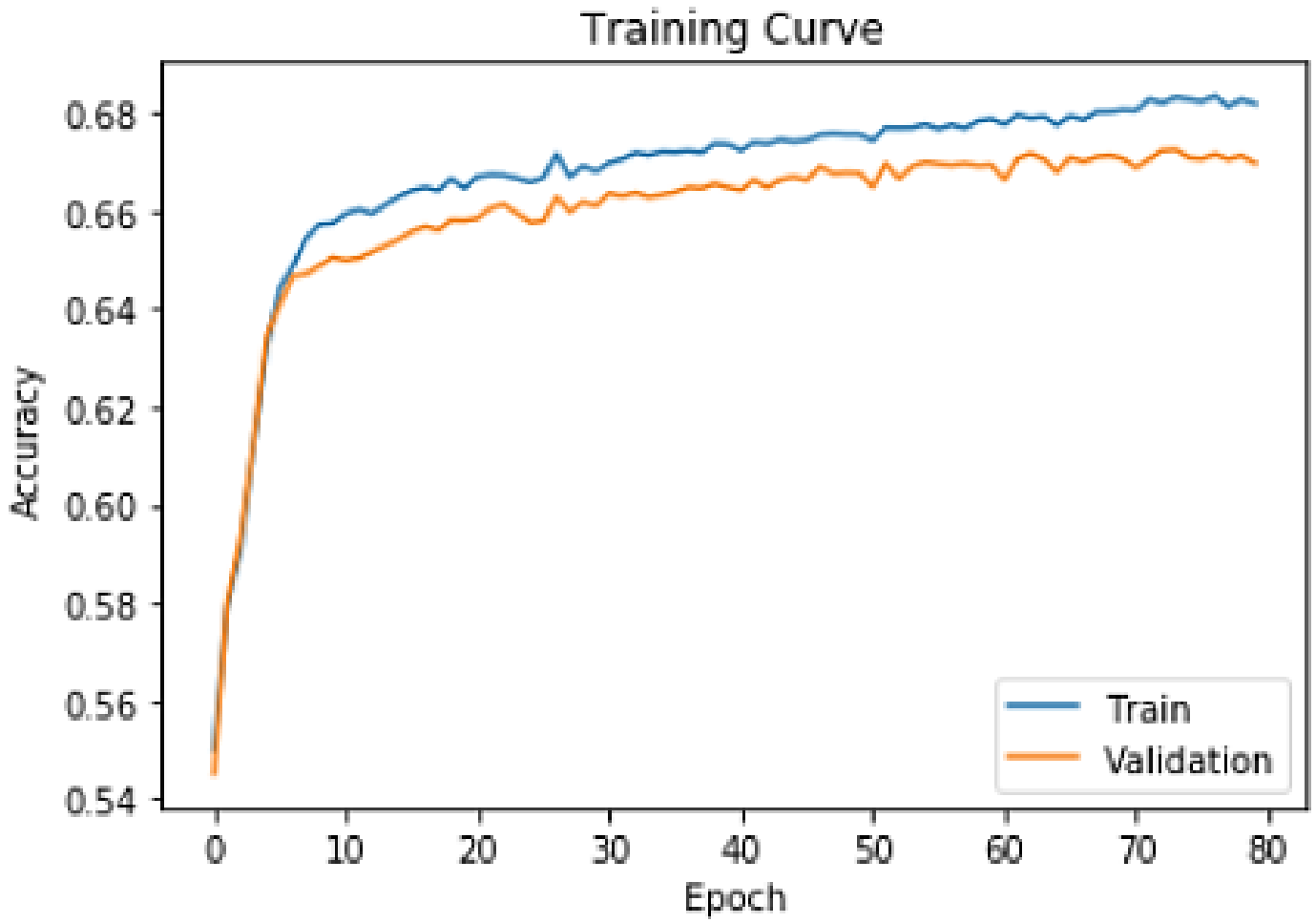
PyTorch Implementation: LSTM

```
class TweetLSTM(nn.Module):
    def __init__(self, input_size, hidden_size, num_classes):
        super(TweetLSTM, self).__init__()
        self.emb = nn.Embedding.from_pretrained(glove.vectors)
        self.hidden_size = hidden_size
        self.rnn = nn.LSTM(input_size, hidden_size, batch_first=True)
        self.fc = nn.Linear(hidden_size, num_classes)

    def forward(self, x):
        # Look up the embedding
        x = self.emb(x)
        # Set an initial hidden state and cell state
        h0 = torch.zeros(1, x.size(0), self.hidden_size)
        c0 = torch.zeros(1, x.size(0), self.hidden_size)
        # Forward propagate the LSTM
        out, _ = self.rnn(x, (h0, c0))
        # Pass the output of the last time step to the classifier
        out = self.fc(out[:, -1, :])
        return out
```

PyTorch Implementation: GRU

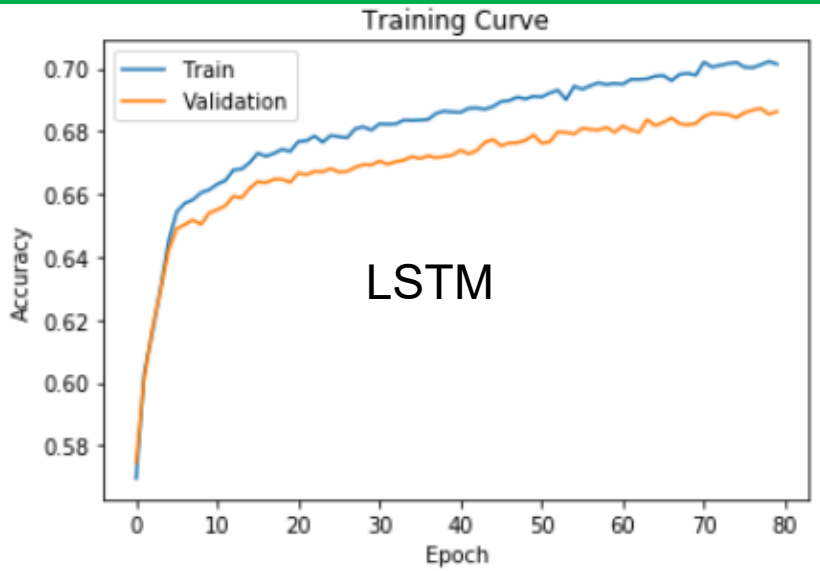
```
class TweetGRU(nn.Module):
    def __init__(self, input_size, hidden_size, num_classes):
        super(TweetGRU, self).__init__()
        self.emb = nn.Embedding.from_pretrained(glove.vectors)
        self.hidden_size = hidden_size
        self.rnn = nn.GRU(input_size, hidden_size, batch_first=True)
        self.fc = nn.Linear(hidden_size, num_classes)

    def forward(self, x):
        # Look up the embedding
        x = self.emb(x)
        # Set an initial hidden state
        h0 = torch.zeros(1, x.size(0), self.hidden_size)
        # Forward propagate the GRU
        out, _ = self.rnn(x, h0)
        # Pass the output of the last time step to the classifier
        out = self.fc(out[:, -1, :])
        return out
```

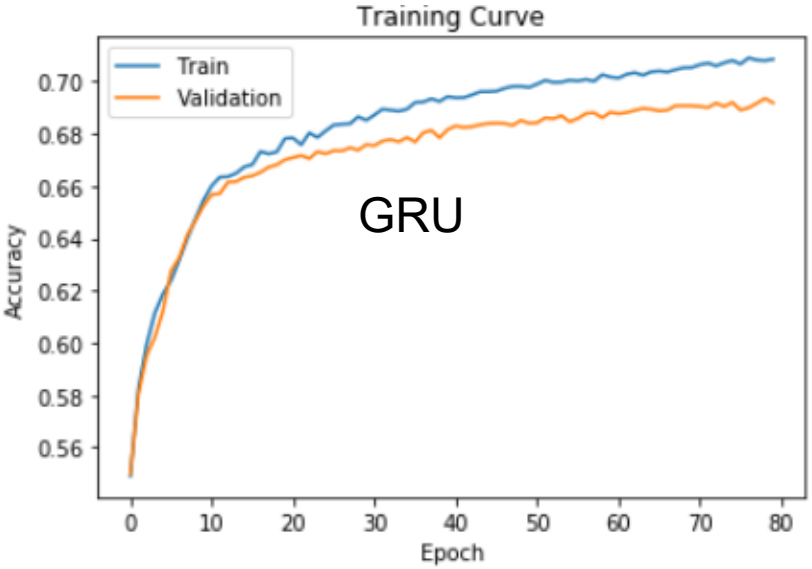
Learning Curves: RNN



Learning Curves: LSTM and GRU

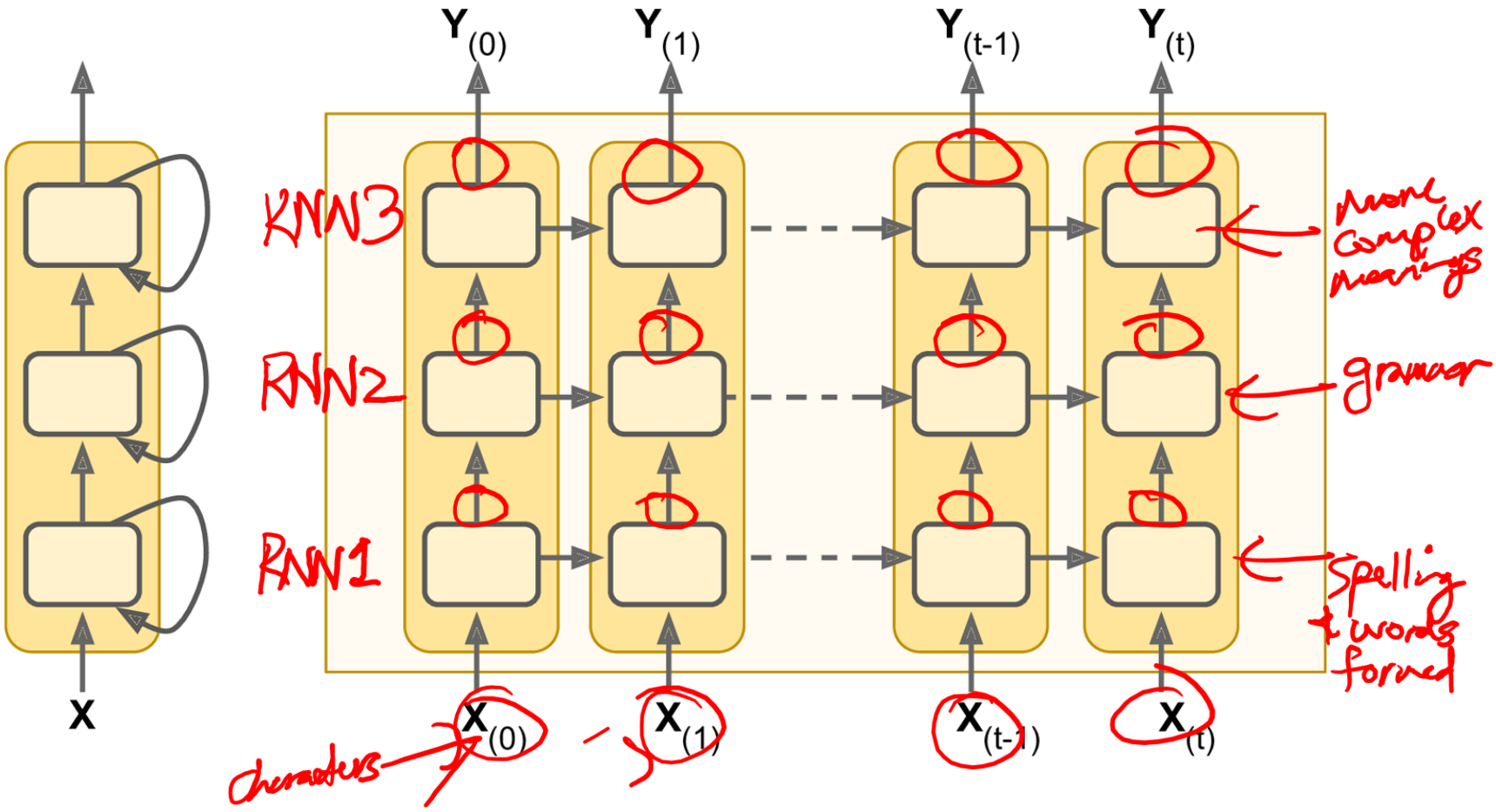
- Learning curves show 1-2% improvement with LSTM and GRU.
- With more fine tuning we should be able to improve on this.



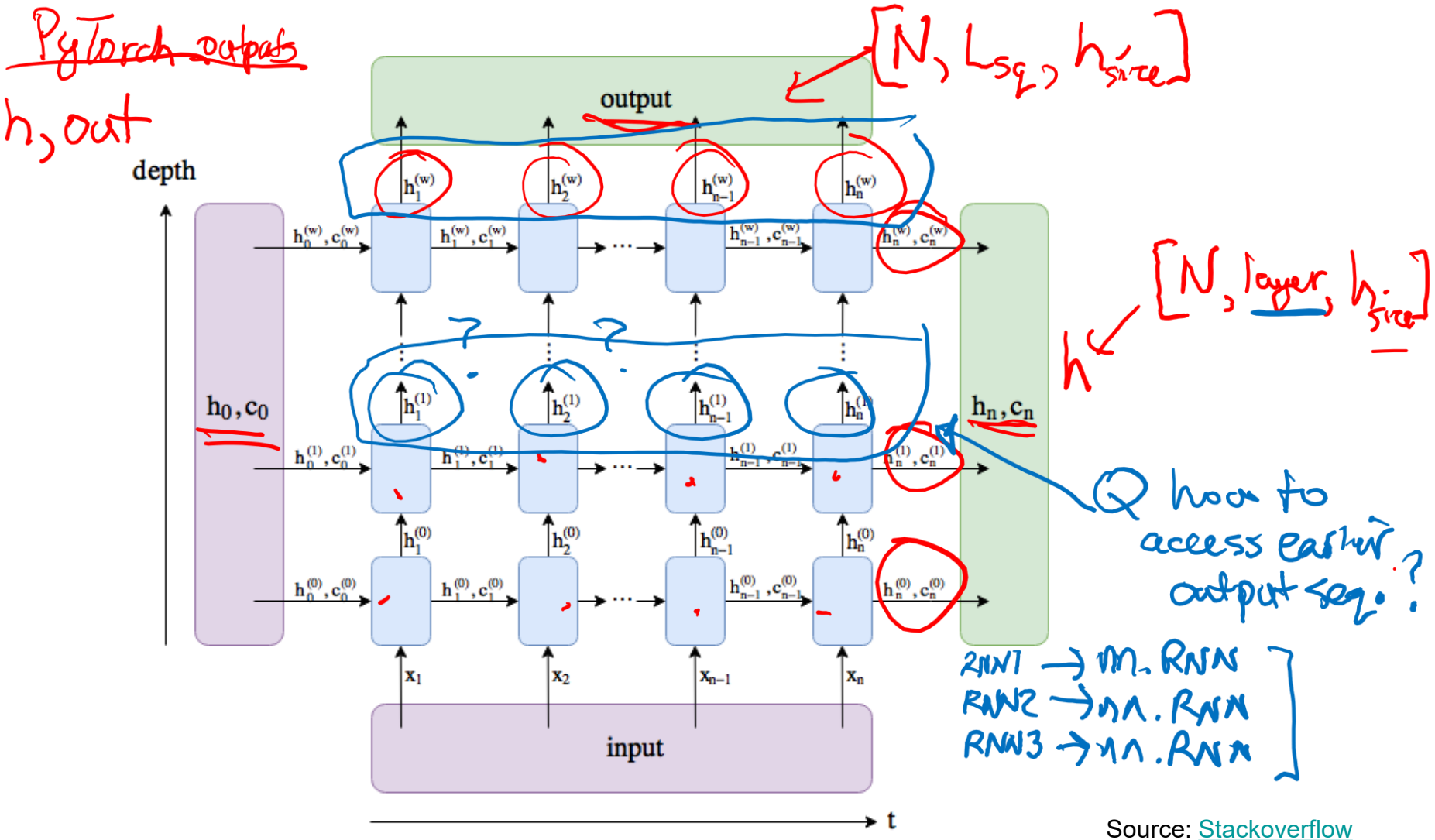


Deep Recurrent Neural Networks

Deep RNNs


Y(0)
Y(1)
Y(t-1)
Y(t)
X
X(0)
X(1)
X(t-1)
X(t)
RNN3
RNN2
RNN1
more complex meanings
grammar
spelling + words formed
characters

Connecting to Downstream Tasks

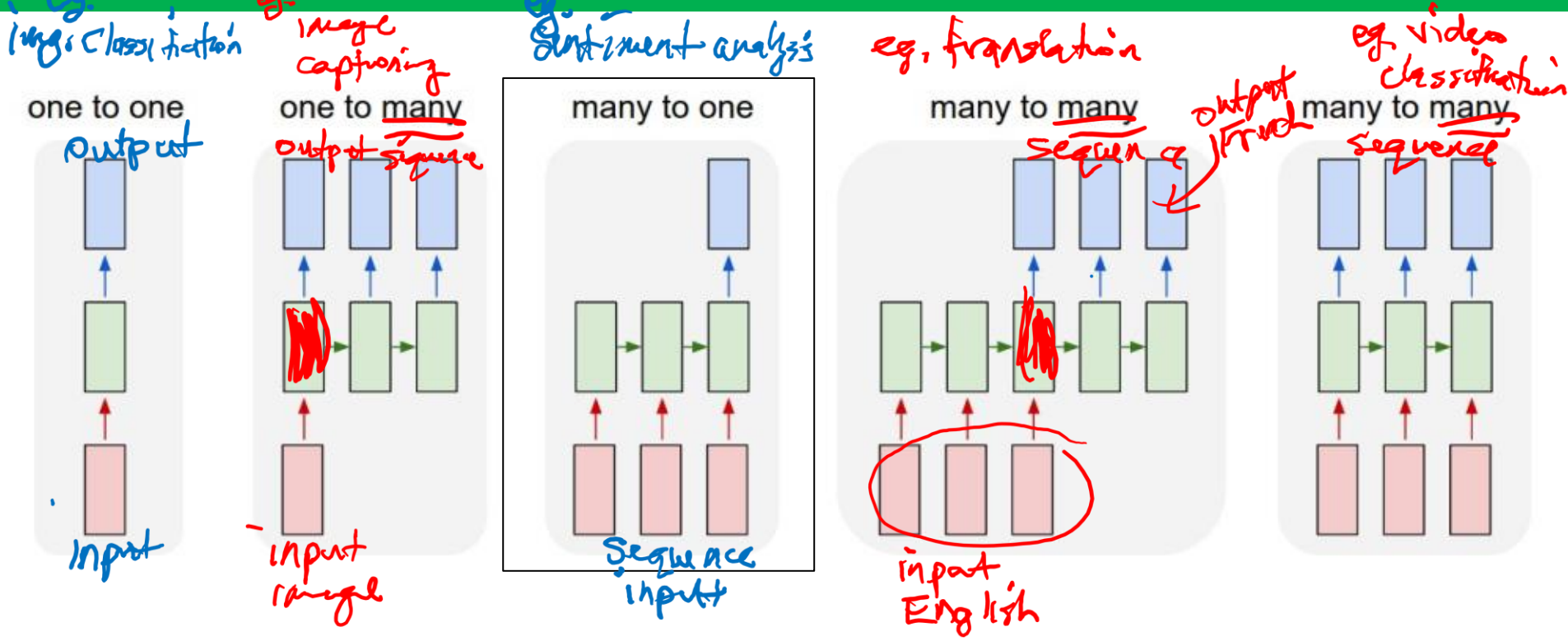


Source: Stackoverflow

Continue at 3:08pm

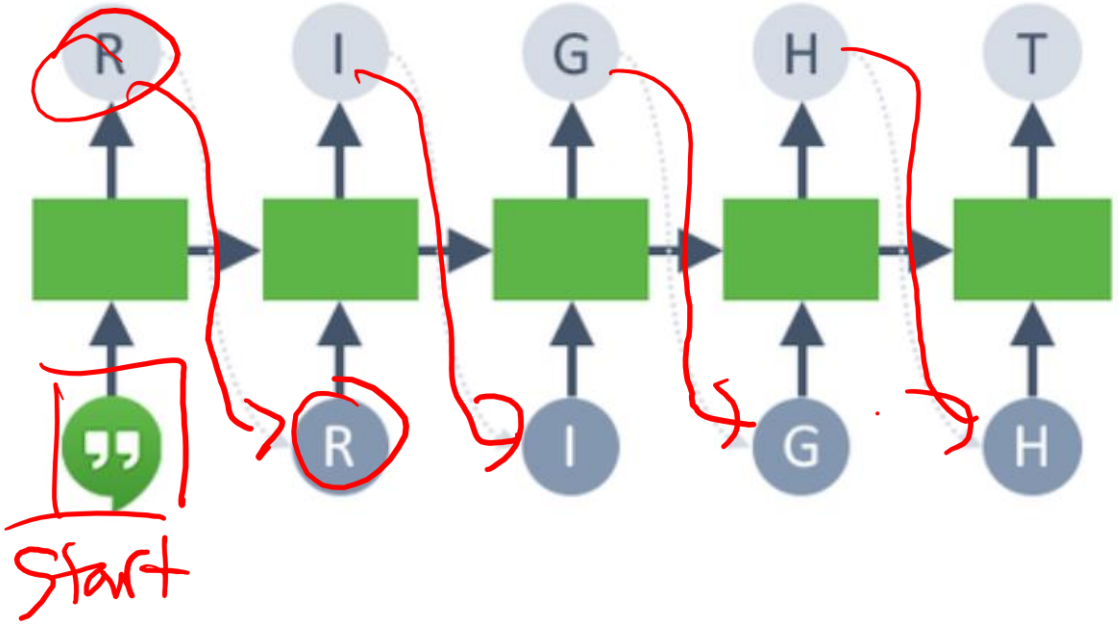
Generating Text

RNN Model Types



- We have focused on **many-to-one** (classification task)
- Other variations available

Generative RNNs

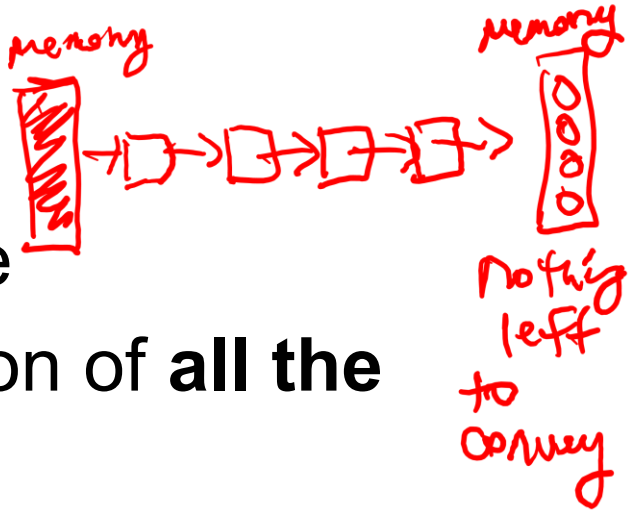


- We have seen how to use RNNs for classification, let us explore how we can use RNNs for generative tasks.

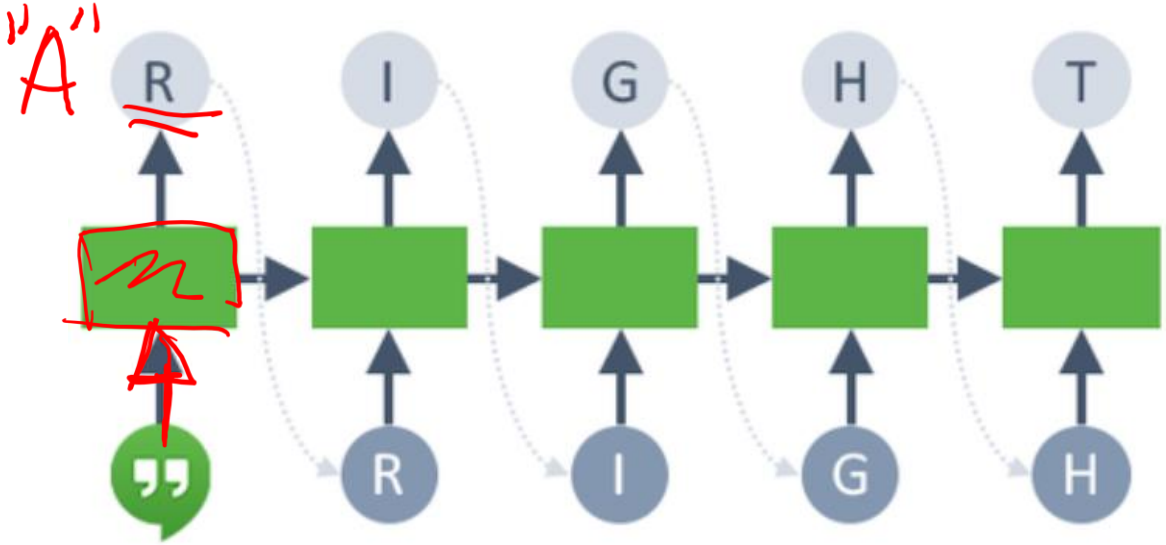
RNN Hidden State Differences

- RNN for **Prediction**:
  - Process tokens one at a time
  - Hidden state is a representation of **all the tokens read thus far**

- RNN for **Generation**:
  - Generate tokens one at a time
  - Hidden state is a representation of **all the tokens to be generated**



RNN for Text Generation



- token = preprocess (input)
- hidden = update_function (hidden, token)
- token_distribution = prediction_function (hidden)
- token = sample_from (token_distribution)

Training Generative RNN

- During training, we try to get the RNN to generate one particular sequence in the training set:
- At each time step:
  - token_distribution = prediction_function(hidden)
  - Compare the token_distribution with the actual next token

- Q1: What kind of a problem is this? (regression or classification?)
- Q2: What loss function should we use? cross-entropy loss

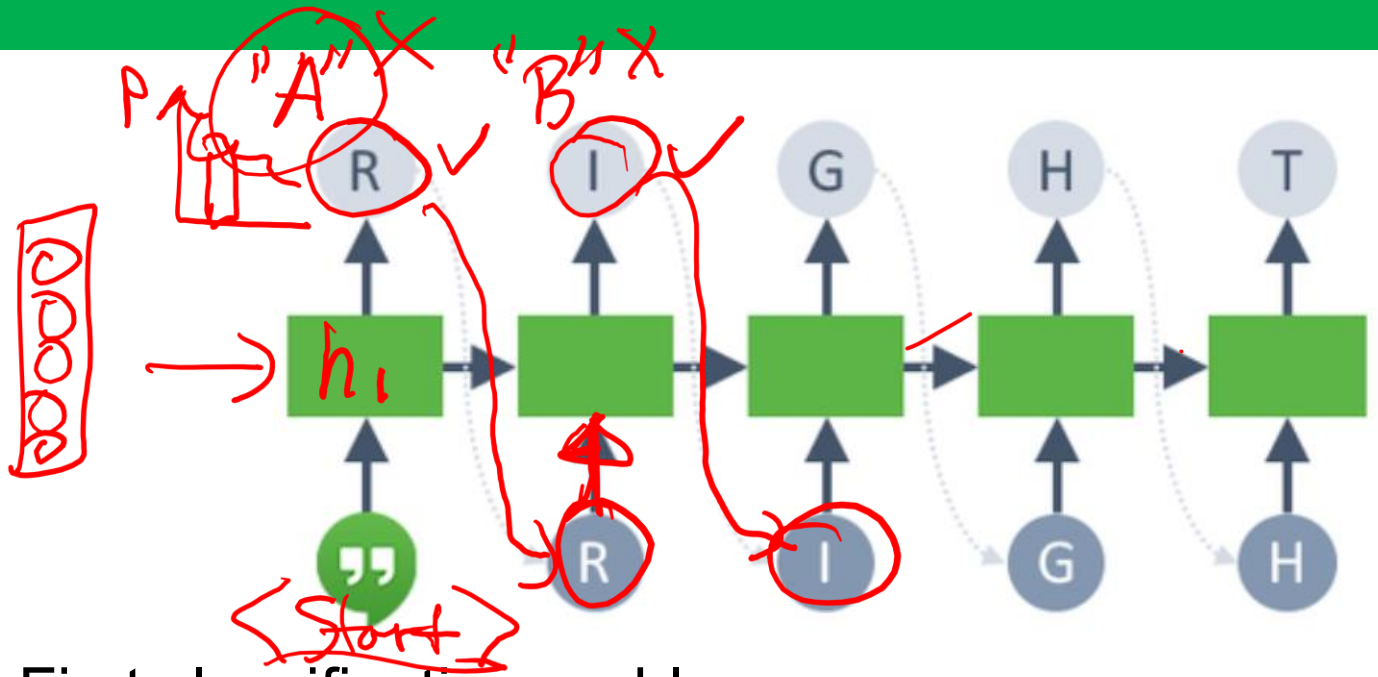
Example: Generate Text

Generative RNNs Con't

Learn to generate new sequences will require some additional changes:

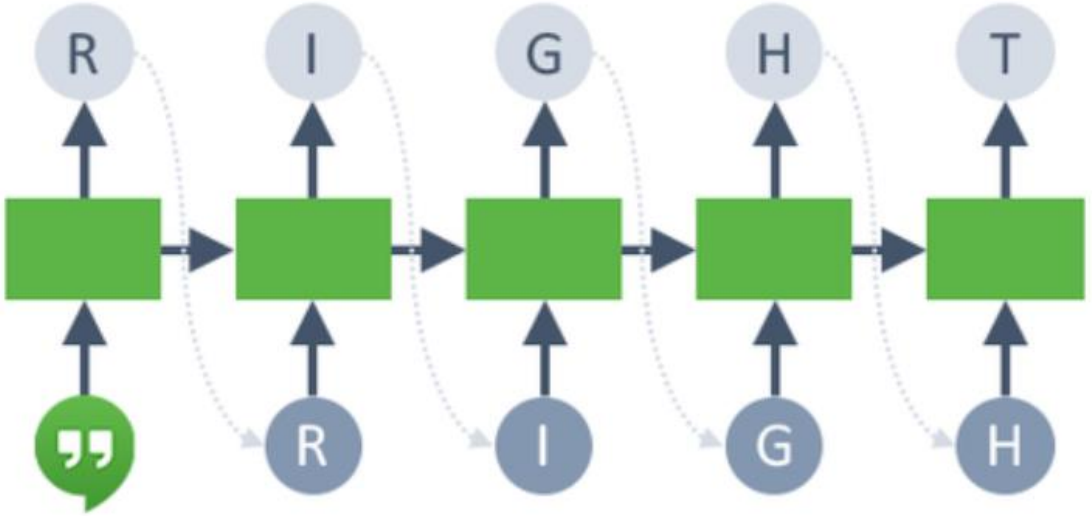
- Input sequence representation
  - Start and End token
- Training-time behaviour
  - Teacher-forcing
- Test-time behaviour
  - Sampling and Temperature

1. Text Generation: Start



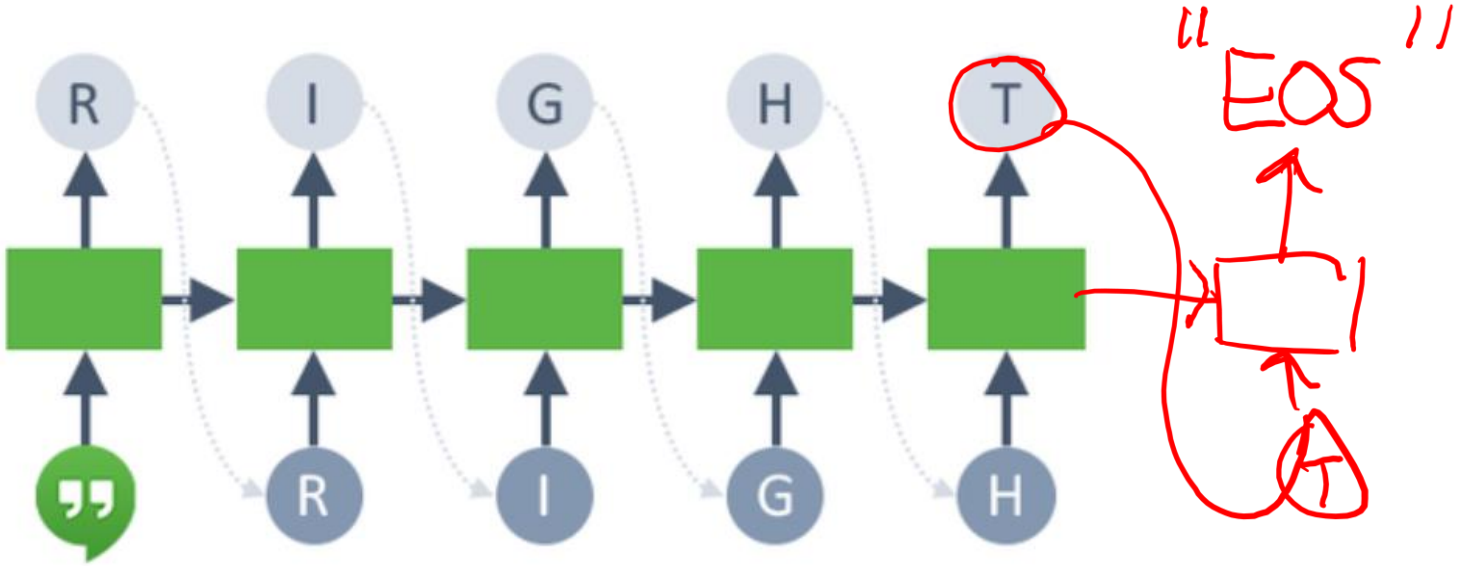
- First classification problem:
  - Start with an initial hidden state
  - **Update the hidden state with a "<BOS>" (beginning of string)** token, so that the hidden state becomes meaningful (not just zeros)
  - Get the distribution over the first character
  - Compute the cross-entropy loss against the ground truth ("R")

2. Text Generation: Teacher Forcing



- Second classification problem:
  - Update the hidden state with the ground truth token (“R”)
  - Regardless of the prediction from the previous step
    - This technique is called **teaching forcing**
  - Get the distribution over the second character
  - Compute the cross-entropy loss against the ground truth (“I”)

3. Text Generation: End



- Continue until we get to the “<EOS>” (end of string) token

Let's Code!

1. Let's look at some data!
2. RNN Architecture
3. RNN Training

Generative RNNs Con't

Learn to generate new sequences will require some changes:

- Input sequence representation
  - Start and End token
- Training-time behaviour
  - Teacher-forcing
- Test-time behaviour
  - Sampling and Temperature

Sampling a Token during Test Time

- Unlike in an actual classification problem, always generating the token with the highest probability won't work well.

- Q: Why?

Sample?

Search Problem

- When using a generative model, we **want to generate new diverse text** that can produce different outputs each time.

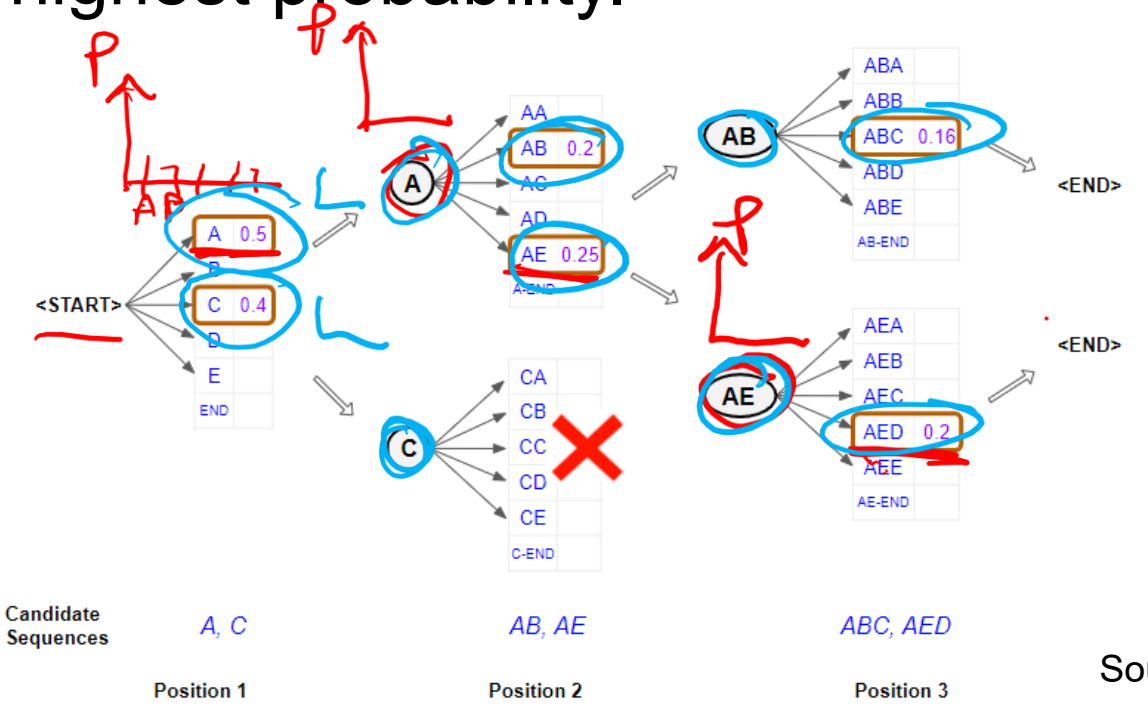
- To achieve this, we **sample from the predicted distributions**. The following are common sampling strategies:

  - Greedy Search
  - Beam Search
  - Temperature

Beam Search is a substitute for Exhaustive Search

Greedy Search vs Beam Search

- **Greedy Search** always selects and generates the token with highest probability.
- **Beam Search** tracks k sequences of tokens with the highest probability.



Q: This is an example of Beam Search. What would Greedy Search look like?

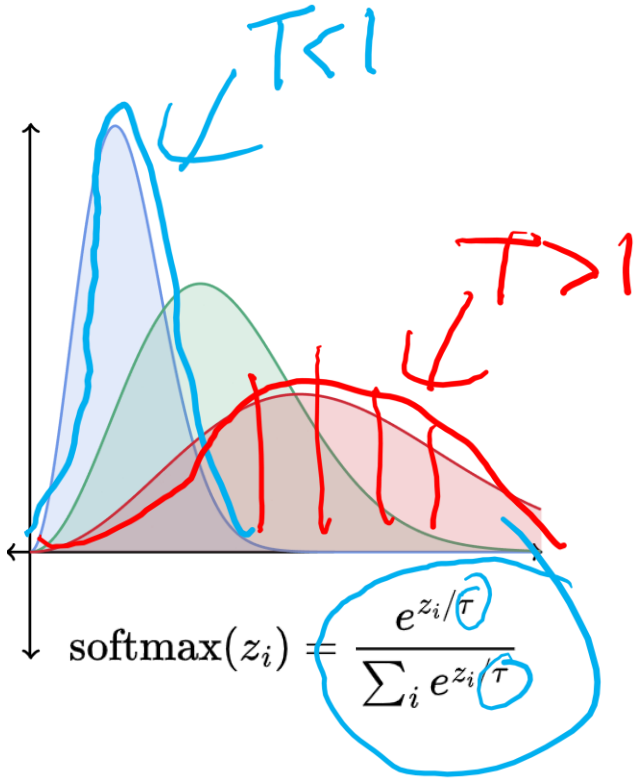
Source: Ketan Doshi

Sampling with Temperature

- Temperature is a test time parameter that allows us to adjust the generated output.

- **Low temperature (T < 1)**:
  - Higher quality samples
  - Less variety / Less random

- **High temperature (T > 1)**:
  - Lower quality samples
  - More variety / More random



Let's Code!

4. Sampling new Text

More RNN Examples

Example: Generate C Code

- Using GitHub open-source data

```
static void do_command(struct seq_file *m, void *v)
{
  int column = 32 << (cmd[2] & 0x80);
  if (state)
    cmd = (int)(int_state ^ (in_8(&ch->ch_flags) & Cmd) ? 2 : 1);
  else
    seq = 1;
  for (i = 0; i < 16; i++) {
    if (k & (1 << 1))
      pipe = (in_use & UMXTHREAD_UNCCA) +
        ((count & 0x00000000fffffff8) & 0x000000f) << 8;
    if (count == 0)
      sub(pid, ppc_md.kexec_handle, 0x20000000);
    pipe_set_bytes(i, 0);
  }
  /* Free our user pages pointer to place camera if all dash */
  subsystem_info = &of_changes[PAGE_SIZE];
  rek_controls(offset, idx, &soffset);
  /* Now we want to deliberately put it to device */
  control_check_polarity(&context, val, 0);
  for (i = 0; i < COUNTER; i++)
    seq_puts(s, "policy ");
}
```

Source: Andrej Karpathy

Example: Algebraic Geometry Textbook

Part	Chapter	online	TeX source	view pdf
Preliminaries				
	1. Introduction	online	tex	pdf
	2. Conventions	online	tex	pdf
	3. Set Theory	online	tex	pdf
	4. Categories	online	tex	pdf
	5. Topology	online	tex	pdf
	6. Sheaves on Spaces	online	tex	pdf
	7. Sites and Sheaves	online	tex	pdf
	8. Stacks	online	tex	pdf
	9. Fields	online	tex	pdf
	10. Commutative Algebra	online	tex	pdf

- The Stacks Project: open-source algebraic geometry textbook
- Latex Source

Source: http://stacks.math.columbia.edu/

Example: Algebraic Geometry Textbook

For $\bigoplus_{n=1,\ldots,m}$ where $\mathcal{L}_{m_\bullet} = 0$, hence we can find a closed subset $\mathcal{H}$ in $\mathcal{H}$ and any sets $\mathcal{F}$ on X, U is a closed immersion of S, then $U \to T$ is a separated algebraic space.

Proof. Proof of (1). It also start we get

$$S = \mathrm{Spec}(R) = U \times_X U \times_X U$$

and the compariciol y in the fibre product covering we have to prove the lemma generated by $\coprod Z \times_U U \to V$. Consider the maps M along the set of points Sch_{fppf} and $U \to U$ is the fibre category of S in U in Section, ?? and the fact that any U affine, see Morphisms, Lemma ??. Hence we obtain a scheme S and any open subset $W \subset U$ in $Sh(G)$ such that $\mathrm{Spec}(R') \to S$ is smooth or an

$$U = \bigcup U_i \times_{S_i} U_i$$

which has a nonzero morphism we may assume that f_i is of finite presentation over S. We claim that $\mathcal{O}_{X,x}$ is a scheme where $x, x', s'' \in S'$ such that $\mathcal{O}_{X,x'} \to \mathcal{O}'_{X',x'}$ is separated. By Algebra, Lemma ?? we can define a map of complexes $\mathrm{GL}_{S'}(x'/S'')$ and we win. □

To prove study we see that $\mathcal{F}|_U$ is a covering of $\mathcal{X}'$, and $\mathcal{T}_i$ is an object of $\mathcal{F}_{X/S}$ for $i > 0$ and $\mathcal{F}_{\bar{p}}$ exists and let $\mathcal{F}_i$ be a presheaf of $\mathcal{O}_X$-modules on $\mathcal{C}$ as a $\mathcal{F}$-module. In particular $\mathcal{F} = U/\mathcal{F}$ we have to show that

$$\widetilde{M}^\bullet = \mathcal{I}^\bullet \otimes_{\mathrm{Spec}(k)} \mathcal{O}_{S,s} - i_X^{-1}\mathcal{F})$$

is a unique morphism of algebraic stacks. Note that

$$\mathrm{Arrows} = (Sch/S)^{opp}_{fppf}, (Sch/S)_{fppf}$$

and

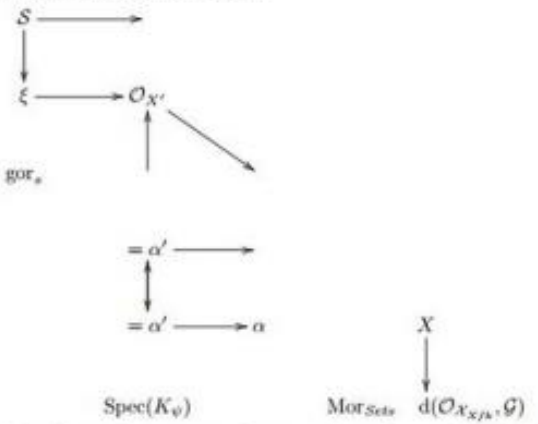
$$V = \Gamma(S, \mathcal{O}) \longmapsto (U, \mathrm{Spec}(A))$$

is an open subset of X. Thus U is affine. This is a continuous map of X is the inverse, the groupoid scheme S.

Proof. See discussion of sheaves of sets. □

The result for prove any open covering follows from the less of Example ??. It may replace S by $X_{spaces,étale}$ which gives an open subspace of X and T equal to S_{Zar}, see Descent, Lemma ??. Namely, by Lemma ?? we see that R is geometrically regular over S.

This since $\mathcal{F} \in \mathcal{F}$ and $x \in \mathcal{G}$ the diagram



is a limit. Then $\mathcal{G}$ is a finite type and assume S is a flat and $\mathcal{F}$ and $\mathcal{G}$ is a finite type $f_\bullet$. This is of finite type diagrams, and

- the composition of $\mathcal{G}$ is a regular sequence,
- $\mathcal{O}_{X'}$ is a sheaf of rings.

□

Proof. We have see that $X = \mathrm{Spec}(R)$ and $\mathcal{F}$ is a finite type representable by algebraic space. The property $\mathcal{F}$ is a finite morphism of algebraic stacks. Then the cohomology of X is an open neighbourhood of U. □

Proof. This is clear that $\mathcal{G}$ is a finite presentation, see Lemmas ??.
A *reduced above* we conclude that U is an open covering of $\mathcal{C}$. The functor $\mathcal{F}$ is a "field

$$\mathcal{O}_{X,x} \longrightarrow \mathcal{F}_{\overline{x}} \quad \text{-1}(\mathcal{O}_{X_{étale}}) \longrightarrow \mathcal{O}_{X_t}^{-1}\mathcal{O}_{X_\lambda}(\mathcal{O}_{X_\eta}^{\nabla})$$

is an isomorphism of covering of $\mathcal{O}_{X_i}$. If $\mathcal{F}$ is the unique element of $\mathcal{F}$ such that X is an isomorphism.
The property $\mathcal{F}$ is a disjoint union of Proposition ?? and we can filtered set of presentations of a scheme $\mathcal{O}_X$-algebra with $\mathcal{F}$ are opens of finite type over S.
If $\mathcal{F}$ is a scheme theoretic image points. □

If $\mathcal{F}$ is a finite direct sum $\mathcal{O}_{X_\lambda}$ is a closed immersion, see Lemma ??. This is a sequence of $\mathcal{F}$ is a similar morphism.

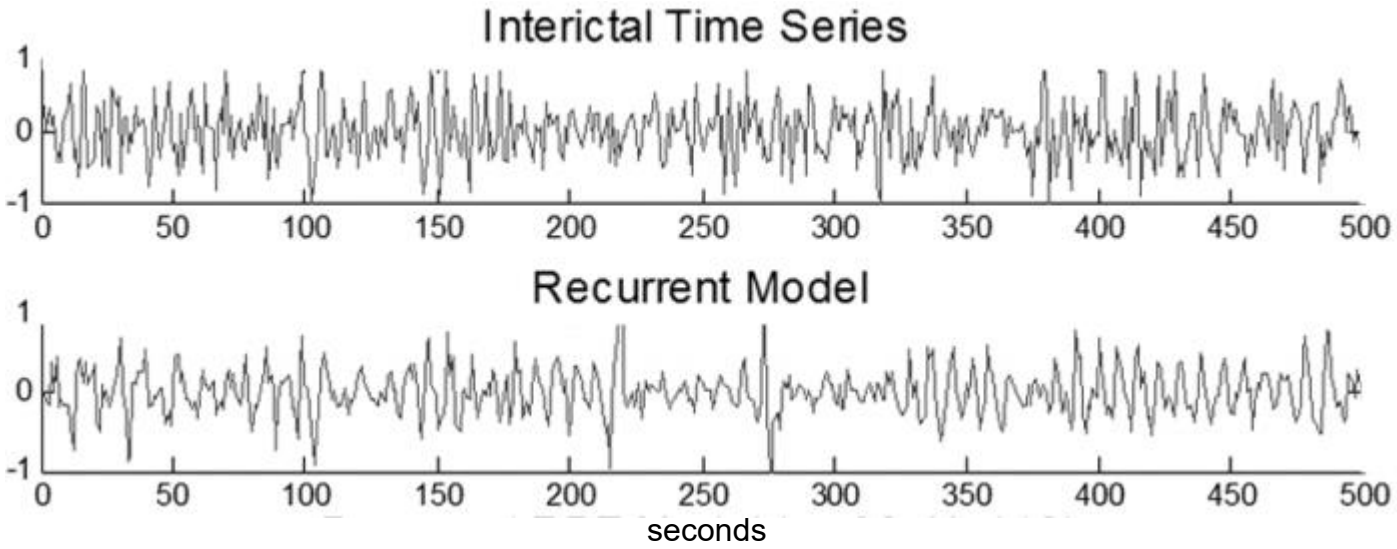
Example: Time-Series Data

- Recurrent Neural Networks to perform time-series forecasting.

Source: Rian Dolphin

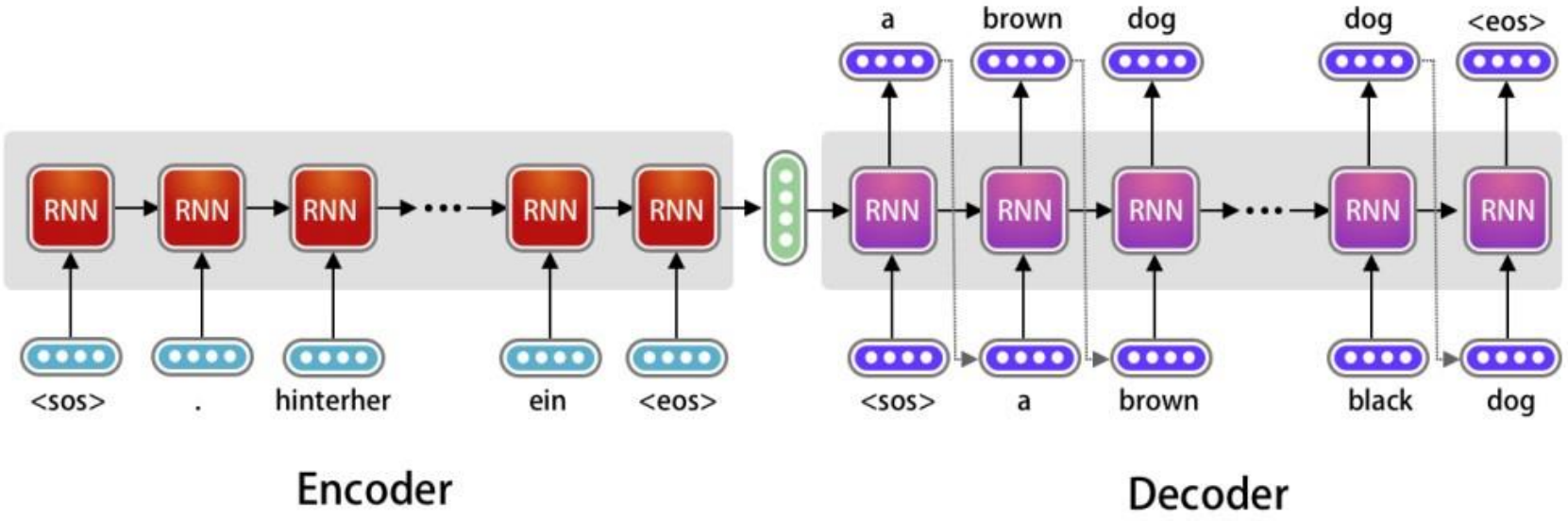
Example: Physiological Signals

- Recurrent Neural Networks to generate a biological-like signal as neuromodulator (i.e. neural stimulator)
- Challenge to mimic the high complexity (chaotic-like) neural activity



Machine Translation

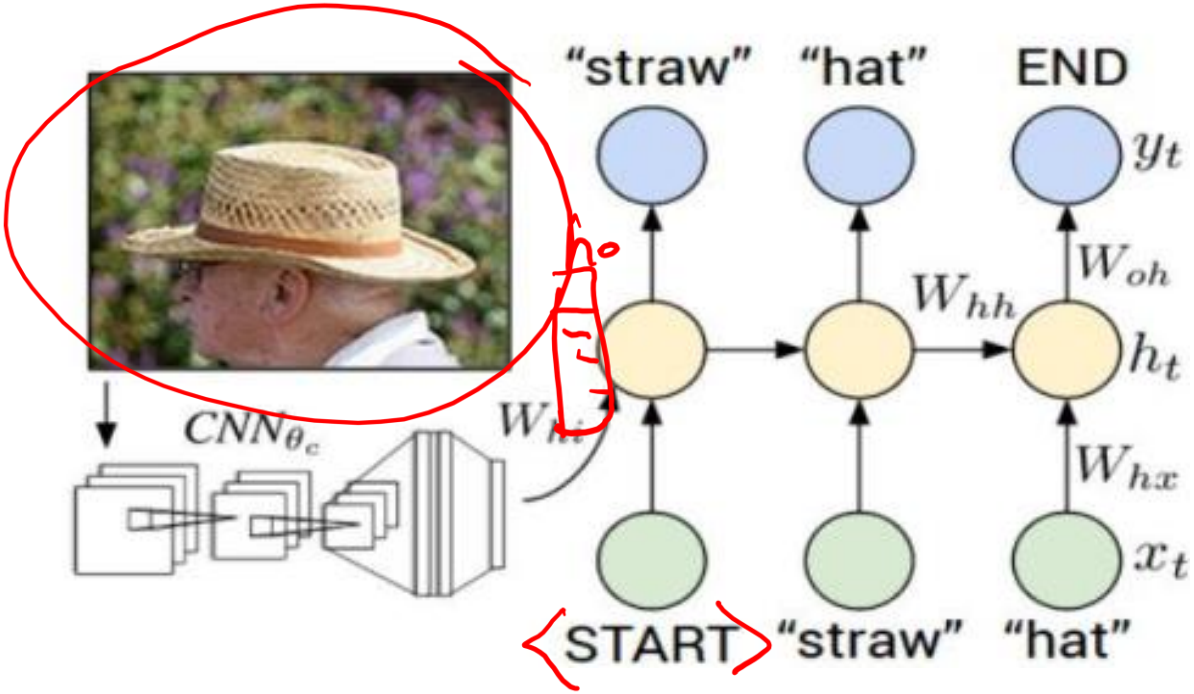
- Sequence to Sequence
  - Sequence to Hidden State
  - Hidden State to Sequence



Source: Abhisek Jana

Combining with CNNs

- Image Captioning



Source: Andrej Karpathy

Combining with CNNs

- Image Captioning: Good Results

A cat sitting on a suitcase on the floor

A cat is sitting on a tree branch

A dog is running in the grass with a frisbee

A white teddy bear sitting in the grass

Two people walking on the beach with surfboards

A tennis player in action on the court

Two giraffes standing in a grassy field

A man riding a dirt bike on a dirt track

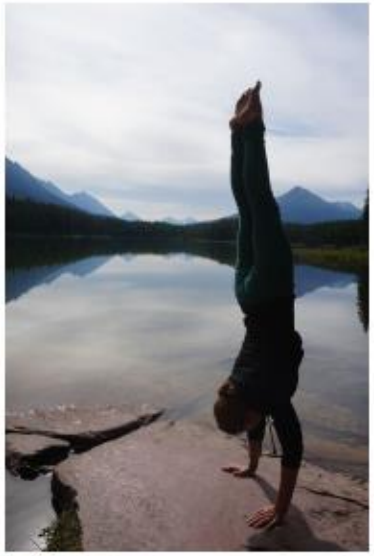
Source: Andrej Karpathy

Combining with CNNs

- Image Captioning: Bad Results

A woman is holding a cat in her hand

A person holding a computer mouse on a desk



A woman standing on a beach holding a surfboard

A bird is perched on a tree branch

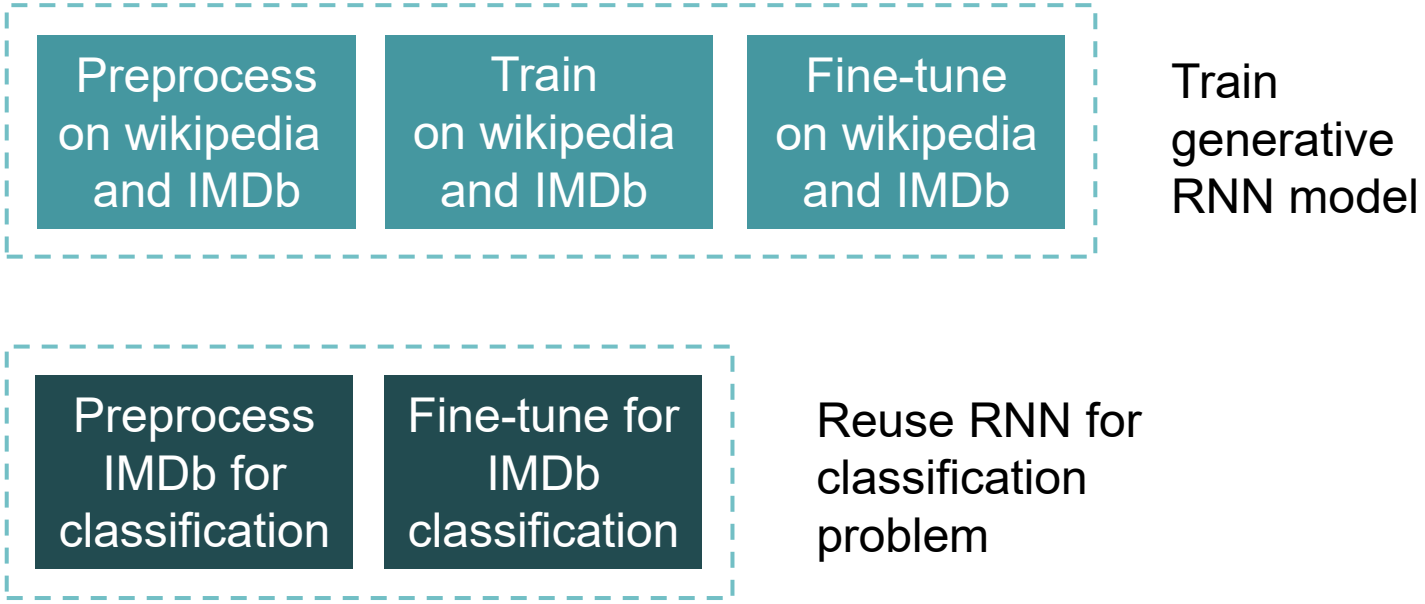
A man in a baseball uniform throwing a ball

Source: Andrej Karpathy

Transfer Learning / Pretrained Models

Transfer Learning for RNNs

- **Common theme in machine learning is transfer learning!** Knowledge gained solving one task can help to solve another task.
- ULMFit (Universal Language Model Fine-Turning)



Bi-directionality

ELMo
(RNN)

BERT
(transformer)

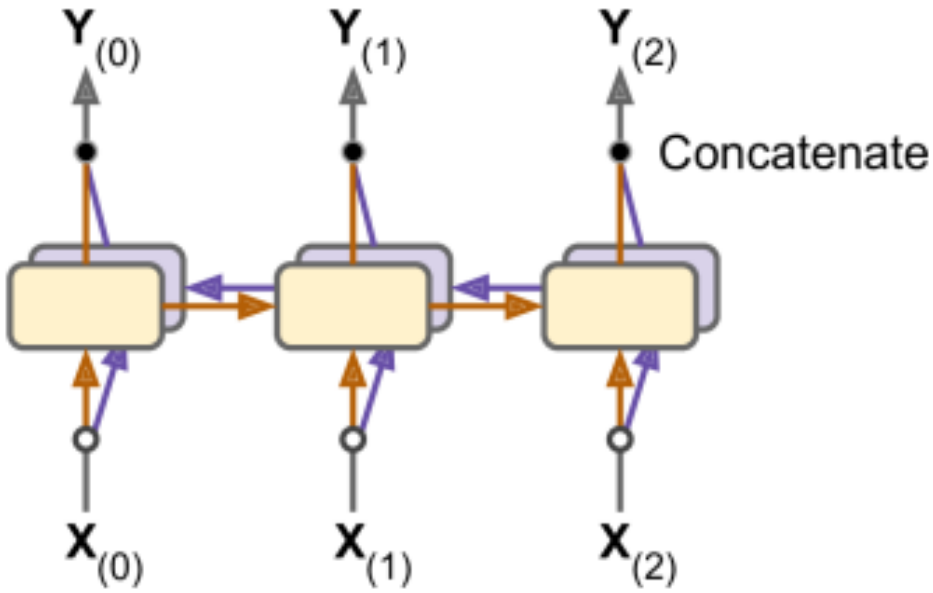
- **Known as encoder models.** Both take advantage of bi-directionality to encode sequential data and lead to better performance.

Bi-Directionality

- Regular sequential models (e.g., RNNs) only looks at past and present inputs before generating their outputs
- These type of models makes sense when forecasting time series, but not for most NLP tasks.
- What does “queen” mean in the following?
  - “the **Queen** of the United Kingdom,” “the **queen** of hearts,” and “the **queen** bee”
- **Q: How can we do better? How can we use future inputs to obtain meaning for “queen”?**

Bidirectional RNNs

- Run two recurrent layers on the same inputs, one reading the words from left to right and the other reading them from right to left
- Then combine (concatenate) their outputs at each time step



Key idea: information earlier or later in the sentence can help disambiguate a word, so we need both directions.

Source: A. Geron, Hands-On ML Textbook

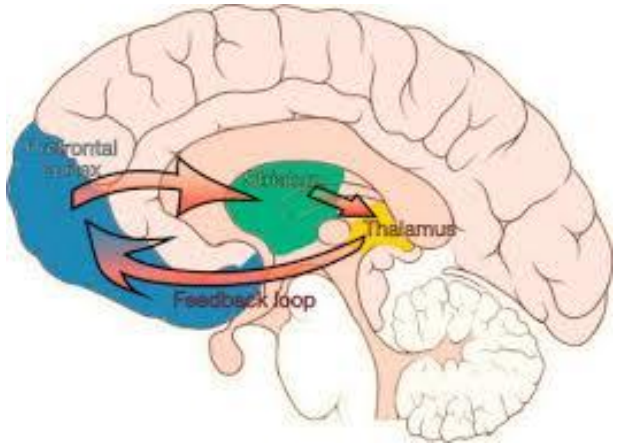
Summary

- RNN Applications
  - Sequential Data
  - Classification Problems
  - Generative Problems
- Architectures
  - RNN, LSTM, GRU
- Large part of the challenge involves preprocessing text data

Biological Motivation

Several types of neural connections have been observed in biology

- Feed-forward
  - Simple ANN, CNN
- Lateral
  - RNNs
- Feed-back
  - Have not been fully replicated in neural networks
  - Difficult to train and may need methods other than backpropagation



Week 10 - Announcements

Assignment 4 – Sequential Data

- The final assignment
- Due Tue, Nov 18th at 11pm

- Part 1: Train an RNN to identify sentiment in movie reviews
- Part 2: Use a pretrained transformer model to identify sentiment in movie reviews

- Bonus: Explore generative LSTM to build a small language model to improve movie reviews. This is like the transfer learning seen before.

Final Exam Details

- When:
  - Start Time: Wednesday, Dec 3 at 1:10pm
  - End Time: Wednesday, Dec 3 at 3:40pm
- Duration:
  - 2.5 hours expected
- Location:
  - MB 128 (Lassonde Mining Building)
- Review:
  - Final exam topics, sample problems and sample final exam will be posted on Quercus

Final Exam Content

- Lectures, Assignments, Preparation Materials
- Topics:
  - Machine Learning Overview
  - Biological Motivations
  - Architectures of ANNs, CNNs, Autoencoders, GANs, RNNs, Transformers, Style Transfer, R-CNN, and more
  - Convolutions and Convolutional Transpose
  - Optimization and Hyperparameters
  - Regularization
  - Word Embeddings
  - Reinforcement Learning
  - Ethics and Fairness

Project Deliverables

Details on project deliverables have been posted on Quercus:

1. Video Presentations due Nov 30th at 11pm
2. Final Project Report due Dec 7th at 11pm

Presentations

- Due at 11pm on Sunday, Nov 30
- Presentations will take place:
  - Option 1: Monday (Dec 1) 9am – 11:30am (2.5 Hrs)
  - Option 2: Monday (Dec 1) 8:30pm – 11:00pm (2.5 Hrs)
  - Option 3: Monday (Dec 1) 9am – 10:15am (1.25 Hrs)
    Monday (Dec 1) 8:30pm – 9:45pm (1.25 Hrs)
- Presentations will take place over Zoom
- Grading will be done by teaching team (75%) and your classmates (25%).

Next Time

Homework

- Review Pre4B Sample Code

Q/A Support Session

- Assignments 4, Project

Lecture Week 11

- Attention Mechanisms
- Transformers